Clustering and Triangle Motifs

EdgeBasedModels.jl Vignette 08

Simon Frost

2026-05-14

Table of contents

Introduction

Real contact networks exhibit clustering — the tendency for two people who share a mutual contact to also be connected to each other, forming triangles. This property, also called triadic closure, is ubiquitous in social networks but absent from the standard configuration model in the large-N limit.

Clustering has important epidemiological consequences. When transmission can travel around a triangle, one of the three edges is "redundant": if A infects B and B infects C, the edge from A to C cannot cause a *new* infection. This correlation means that clustering reduces epidemic severity compared to a tree-like network with the same degree distribution.

The edge-based compartmental model framework handles clustering by introducing a bivariate probability generating function $g(x, y)$ where x tracks single-edge stubs and y tracks triangle-edge stubs. This approach follows Volz (2011) and Miller (2009).

In this vignette we show how to:

1. Build clustered network models using `ClusteredPGF`
2. Compare clustered and unclustered epidemics
3. Explore how increasing clustering reduces R_0 and epidemic severity

Setup

```
using EdgeBasedModels
using ModelingToolkit
using OrdinaryDiffEq
using Symbolics
using Plots
```

Clustered networks

In a clustered network each node has two types of half-edges:

- Single-edge stubs (s): connect to a random partner (tree-like)
- Triangle-edge stubs (t): connect in groups of three to form triangles

The joint degree distribution is encoded by a bivariate PGF:

$$g(x, y) = \sum_{s,t} p_{s,t}\, x^s\, y^t$$

where $p_{s,t}$ is the probability that a node has s single-edge stubs and t triangle-edge stubs. The mean single degree is $\langle s \rangle = g_x(1, 1)$ and the mean triangle degree is $\langle t \rangle = g_y(1, 1)$.

The clustering coefficient measures the fraction of connected triples that are closed into triangles:

$$C = \frac{2\langle t \rangle}{2\langle t \rangle + \langle s \rangle}$$

For a Poisson clustered network with mean single-edge degree κ_s and mean triangle-edge degree κ_t:

$$g(x, y) = e^{\kappa_s(x-1)+\kappa_t(y-1)}$$

```julia
κ_s = 3.0
κ_t = 1.0
cpgf = clustered_poisson_pgf(κ_s, κ_t)
println("Mean single-edge degree: ", mean_single_degree(cpgf))
println("Mean triangle-edge degree: ", mean_triangle_degree(cpgf))
println("Clustering coefficient: ", clustering_coefficient(cpgf))
```

```
Mean single-edge degree: 3.0exp(0.0)
Mean triangle-edge degree: exp(0.0)
Clustering coefficient: 0.4
```

Each triangle-edge stub participates in one triangle with two partners, so the total mean degree (number of distinct neighbours) is $\langle s \rangle + 2\langle t \rangle$.

Building a clustered SIR model

The key insight of the clustered EBCM is that we need two survival probabilities:

- $\theta_2(t)$: probability a single-edge partner has not transmitted infection
- $\theta_3(t)$: probability a triangle-edge partner has not transmitted infection

The susceptible fraction is then $S(t) = g(\theta_2, \theta_3^2)$, where the square on θ_3 accounts for the two triangle partners per triangle stub.

Let us build both an unclustered and a clustered SIR model with comparable mean degrees.

```julia
@parameters β γ

# Universal anchors: γ=0.25, R₀=2, β derived per scenario (see plan.md)
γ_val = 0.25
R0_target = 2.0
κ_total = 5.0
T_unclustered = R0_target / κ_total
β_val = T_unclustered * γ_val / (1 - T_unclustered)

# Unclustered: all edges are single-stubs, mean degree 5
pgf_unclustered = poisson_pgf(κ_total)
model_unclustered = build_sir(pgf_unclustered, β, γ; form = :expanded)

# Clustered: κ_s=3, κ_t=1 → total mean degree = 3 + 2(1) = 5
cpgf = clustered_poisson_pgf(3.0, 1.0)
model_clustered = build_clustered_sir(cpgf, β, γ)
```

```
EdgeModelSystem(Model clustered_sir:
Equations (8):
  8 standard: see equations(clustered_sir)
Unknowns (8): see unknowns(clustered_sir)
  pop_R(t)
  pop_I(t)
  φ3_R(t)
  φ3_I(t)
  ⍰
Parameters (3): see parameters(clustered_sir)
  ρ
  β
  γ
Observed (4): see observed(clustered_sir), Dict{Symbol, Any}(:θ₃ => θ₃(t), :φ3_I => φ3_I(t), :
1 + 3.0(-1 + θ₂(t)) + θ₃(t)^2))], :rho_param => ρ, :edge_seed_groups => Any[(entry = φ2_I(t), t
1 + 3.0(-1 + θ₂(t)) + θ₃(t)^2) - exp(-1 + 3.0(-1 + θ₂(t)) + θ₃(t)^2)*ρ) / exp(0.0)), (entry = φ3_
1 + 3.0(-1 + θ₂(t)) + θ₃(t)^2)*(1 - ρ)) / exp(0.0))]))
```

The clustered model tracks more state variables due to the separate single-edge and triangle-edge dynamics:

```
println("Unclustered variables: ", keys(model_unclustered.variables))
println("Clustered variables:   ", keys(model_clustered.variables))
```

```
Unclustered variables: [:R, :φ_I, :pop_I, :pop_R, :φ_R, :θ]
Clustered variables:  [:θ₃, :φ3_I, :φ3_R, :φ2_I, :pop_I, :R, :pop_R, :φ2_R, :θ₂]
```

Comparing clustered vs unclustered epidemics

We now solve both models with the unclustered Poisson baseline anchored at $R_0 = 2$ ($\beta = 1/6$, $\gamma = 0.25$), and compare the epidemic curves.

```
tspan = (0.0, 40.0)
params = Dict(β ⇒ β_val, γ ⇒ γ_val)

# Unclustered
ic1 = merge(default_initial_conditions(model_unclustered), params)
prob1 = ODEProblem(model_unclustered.system, ic1, tspan)
sol1 = solve(prob1, Tsit5(); saveat = 0.5)

# Clustered
```

```julia
ic2 = merge(default_initial_conditions(model_clustered), params)
prob2 = ODEProblem(model_clustered.system, ic2, tspan)
sol2 = solve(prob2, Tsit5(); saveat = 0.5)
```

```
retcode: Success
Interpolation: 1st order linear
t: 81-element Vector{Float64}:
  0.0
  0.5
  1.0
  1.5
  2.0
  2.5
  3.0
  3.5
  4.0
  4.5
  ⋮
 36.0
 36.5
 37.0
 37.5
 38.0
 38.5
 39.0
 39.5
 40.0
u: 81-element Vector{Vector{Float64}}:
 [0.0, 0.001, 0.0, 0.0010000000000000009, 0.0, 0.0010000000000000009, 1.0, 1.0]
 [0.000144623672614379, 0.001326107805807947, 0.00014461643846619562, 0.00132597324705529
 [0.00033497795415506396, 0.001735206916235653, 0.0003349379878670594, 0.001734784626555920
 [0.0005828644053464031, 0.002251032455824332, 0.0005827405493116342, 0.002250055298484182
 [0.0009034147604395979, 0.002903445501269235, 0.0009031116558782131, 0.002901456725756914
 [0.0013159697685541298, 0.003729998128762145, 0.0013153165442927257, 0.003726225967734569
 [0.001845104935954855, 0.004777700564948291, 0.0018438057262388148, 0.004770859951768346, 0
 [0.002522040515334812 3, 0.00610529737504897, 0.0025195880956793057, 0.006093255706997098 6,
 [0.0033860764081957277, 0.007785467336545977, 0.0033816217012115767, 0.007764739705640867,
 [0.004486804708450796, 0.009907869260592578, 0.004478935779073265, 0.009872758398233766, 0.
 ⋮
 [0.7684924194637379, 0.007192733932296931, 0.5103843458746748, 0.0013664247117114415, 0.464
 [0.7693445792642608, 0.006452802328367567, 0.5105445393464301, 0.0011974197922827636, 0.464
 [0.7701087548816637, 0.005786927653077036, 0.5106847513725135, 0.0010495222828236022, 0.464
```

```
[0.7707938267003962, 0.005187961768522474, 0.5108074749911291, 0.0009201003717910553, 0.464
[0.7714078772463105, 0.004649395858668374, 0.5109149871101936, 0.0008067432479673713, 0.465
[0.7719581911297007, 0.004165360386799442, 0.5110093483369407, 0.0007072612939204892, 0.465
[0.7724512024872386, 0.003730549318561273, 0.5110921803987606, 0.0006199402285695773, 0.465
[0.7728926785078177, 0.0033401151108894125, 0.511164784159956, 0.0005434150928230247, 0.465
[0.7732878931740367, 0.002989659767265237, 0.5112284260589087, 0.0004763476695847786 4, 0.4
```

Extract the epidemic compartments using the PGFs:

```
S1 = compartment(sol1, model_unclustered, :S)
I1 = compartment(sol1, model_unclustered, :I)
R1 = compartment(sol1, model_unclustered, :R)

S2 = compartment(sol2, model_clustered, :S)
I2 = compartment(sol2, model_clustered, :I)
R2 = compartment(sol2, model_clustered, :R)
```

```
81-element Vector{Float64}:
 0.0
 0.000144623672614379
 0.00033497795415506396
 0.0005828644053464031
 0.0009034147604395979
 0.0013159697685541298
 0.001845104935954855
 0.0025220405153348123
 0.0033860764081957277
 0.004486804708450796
 ?
 0.7684924194637379
 0.7693445792642608
 0.7701087548816637
 0.7707938267003962
 0.7714078772463105
 0.7719581911297007
 0.7724512024872386
 0.7728926785078177
 0.7732878931740367
```

```
plot(sol1.t, S1, label = "S (unclustered)", linewidth = 2, color = :blue)
plot!(sol1.t, I1, label = "I (unclustered)", linewidth = 2, color = :red)
```

```
plot!(sol1.t, R1, label = "R (unclustered)", linewidth = 2, color = :green)
plot!(sol2.t, S2, label = "S (clustered)", linewidth = 2, color = :blue, linestyle = :dash
plot!(sol2.t, I2, label = "I (clustered)", linewidth = 2, color = :red, linestyle = :dash)
plot!(sol2.t, R2, label = "R (clustered)", linewidth = 2, color = :green, linestyle = :das
xlabel!("Time")
ylabel!("Fraction of population")
title!("Clustered vs Unclustered SIR (R₀=2 unclustered baseline)")
```

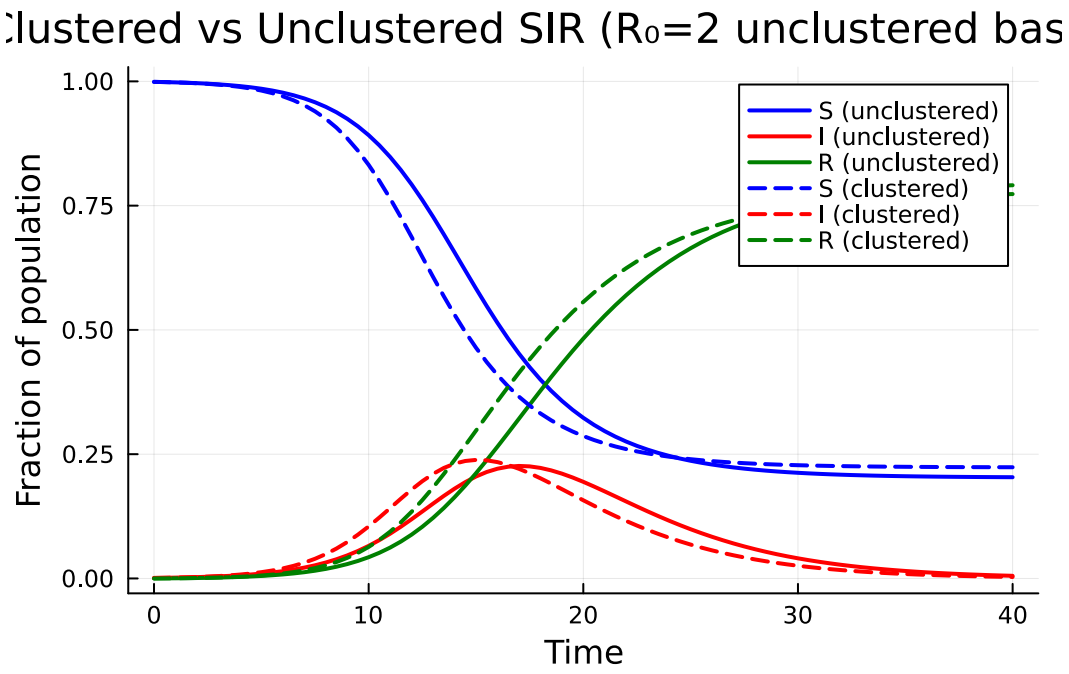

Figure 1: Clustering reduces peak prevalence and delays the epidemic peak. Both networks have mean degree 5.

The dashed (clustered) epidemic has a lower peak prevalence and a delayed peak compared to the solid (unclustered) curves. The final epidemic size is also smaller with clustering.

Varying the clustering coefficient

We fix the total mean degree at 5 and vary the fraction of edges that come from triangles. As clustering increases, more edges are "wasted" on redundant triangle paths.

```
fracs = [0.0, 0.2, 0.4, 0.6]
results = []

for frac in fracs
    κ_s_i = 5.0 * (1 - frac)
    κ_t_i = 5.0 * frac / 2  # each triangle stub contributes 2 neighbours
    cpgf_i = clustered_poisson_pgf(κ_s_i, κ_t_i)
    model_i = build_clustered_sir(cpgf_i, β, γ)
    ic_i = merge(default_initial_conditions(model_i), params)
    prob_i = ODEProblem(model_i.system, ic_i, tspan)
    sol_i = solve(prob_i, Tsit5(); saveat = 0.5)

    S_i = compartment(sol_i, model_i, :S)
    I_i = compartment(sol_i, model_i, :I)
    R_i_vals = compartment(sol_i, model_i, :R)
    C_i = 2κ_t_i / (2κ_t_i + κ_s_i)

    push!(results, (frac = frac, C = C_i, t = sol_i.t, I = I_i, S = S_i, R = R_i_vals))
end
```

```
p = plot(xlabel = "Time", ylabel = "Fraction infected",
         title = "Effect of clustering on epidemic dynamics")
colors = [:red, :orange, :purple, :blue]
for (i, res) in enumerate(results)
    plot!(p, res.t, res.I,
          label = "C = $(round(res.C; digits=2)) (frac = $(res.frac))",
          linewidth = 2, color = colors[i])
end
p
```

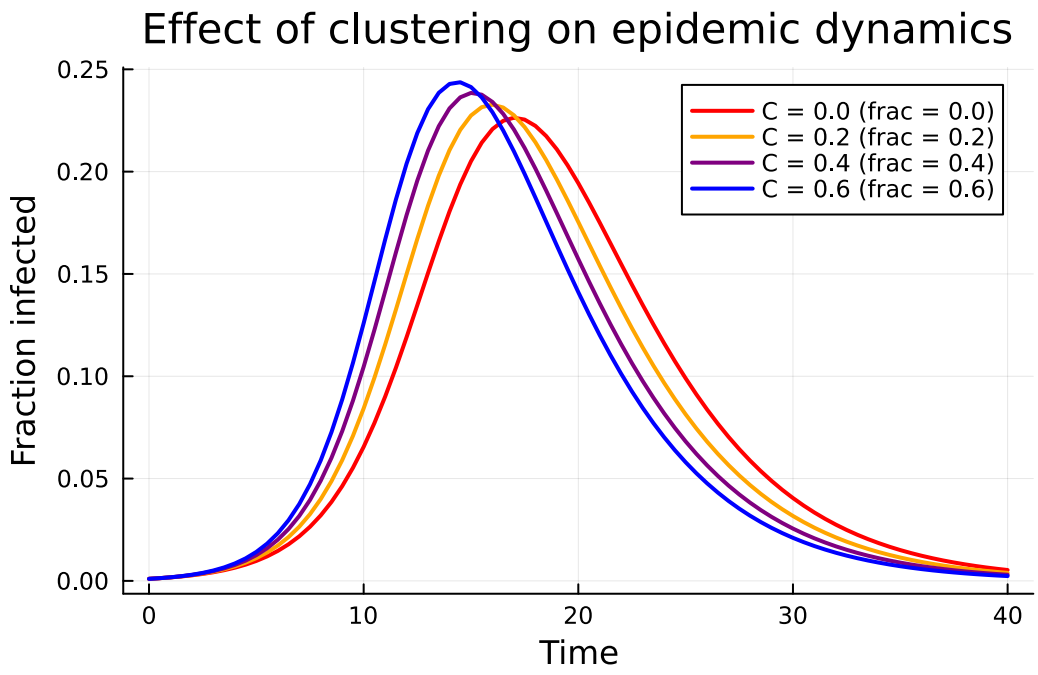

Figure 2: Increasing the fraction of triangle edges reduces epidemic severity while keeping total mean degree = 5.

As the clustering coefficient C increases, the peak prevalence decreases and the epidemic is delayed.

Clustering coefficient and R_0

The basic reproduction number R_0 for a clustered network accounts for the redundant transmission paths within triangles. We can compute it using `basic_reproduction_number`:

```
using EdgeBasedModels: ClusteredConfigurationModel, DiseaseProgression, DiseaseStage, Dise

fracs_fine = 0.0:0.05:0.8
C_vals = Float64[]
R0_vals = Float64[]

for frac in fracs_fine
    κ_s_i = 5.0 * (1 - frac)
    κ_t_i = 5.0 * frac / 2
    cpgf_i = clustered_poisson_pgf(κ_s_i, κ_t_i)
    C_i = 2κ_t_i / (2κ_t_i + κ_s_i)
```

```
    push!(C_vals, C_i)

    prog = DiseaseProgression(
        [DiseaseStage(:I; transmission_rate = β_val),
         DiseaseStage(:R; transmission_rate = 0)],
        [DiseaseTransition(:I, :R, γ_val)]; entry = :I)
    cm = ClusteredConfigurationModel(cpgf_i, prog)
    R0_i = basic_reproduction_number(cm)
    push!(R0_vals, Float64(Symbolics.value(R0_i)))
end
```

```
plot(C_vals, R0_vals, linewidth = 2, color = :darkred, marker = :circle, markersize = 3,
     label = "R₀")
xlabel!("Clustering coefficient C")
ylabel!("R₀")
title!("R₀ vs Clustering (total mean degree = 5, anchored β, γ)")
hline!([1.0], linestyle = :dash, color = :gray, label = "R₀ = 1")
```

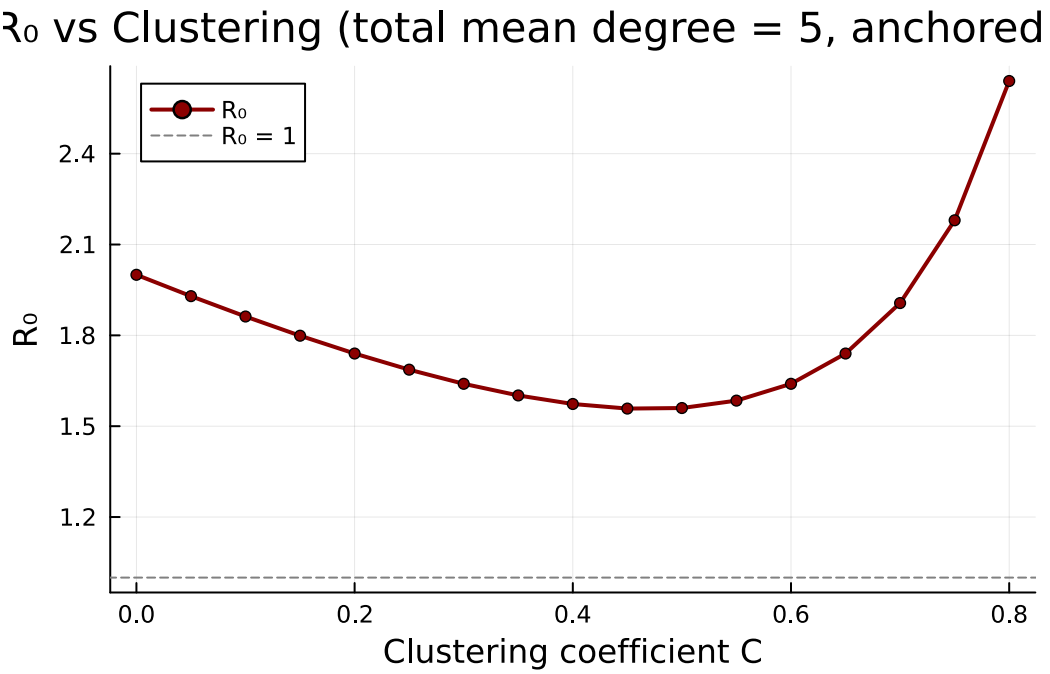

Figure 3: R_0 decreases as the clustering coefficient increases, reflecting the redundancy of triangle edges.

As the clustering coefficient rises from 0 toward 1, R_0 decreases. This quantifies the protective

effect of clustering: triangles create correlated transmission paths that reduce the effective number of secondary infections.

SEIR with clustering

The clustered EBCM framework extends to arbitrary disease progressions. Here we add a latent (exposed) period using `build_clustered_seir`:

```
@parameters σ

cpgf_seir = clustered_poisson_pgf(3.0, 1.0)
model_seir = build_clustered_seir(cpgf_seir, σ, β, γ)
println("SEIR clustered variables: ", keys(model_seir.variables))
```

```
SEIR clustered variables: [:θ₃, :φ3_I, :φ3_R, :φ2_I, :pop_I, :R, :θ₂, :φ2_E, :φ3_E, :pop_R, :φ
```

```
params_seir = Dict(β ⇒ β_val, γ ⇒ γ_val, σ ⇒ 0.2)
ic_seir = merge(default_initial_conditions(model_seir), params_seir)
prob_seir = ODEProblem(model_seir.system, ic_seir, tspan)
sol_seir = solve(prob_seir, Tsit5(); saveat = 0.5)

S_seir = compartment(sol_seir, model_seir, :S)
R_seir_vals = compartment(sol_seir, model_seir, :R)
I_seir = 1.0 .- S_seir .- R_seir_vals
```

```
81-element Vector{Float64}:
 0.0010000000000000009
 0.0010130454905676778
 0.00104717512584005
 0.0010968014650383033
 0.0011582546048606796
 0.0012291684860312529
 0.0013080987956067896
 0.0013942005315523895
 0.0014870910042569122
 0.0015866444132882913
 ⋮
 0.07593289690599012
 0.07978103926137754
 0.08374450570741221
```

```
0.08781939417069416
0.09200168706017353
0.09628717125938369
0.10066838862572111
0.10513033548489154
0.10966182705534845
```

```
plot(sol_seir.t, S_seir, label = "S", linewidth = 2, color = :blue)
plot!(sol_seir.t, I_seir, label = "E + I", linewidth = 2, color = :red)
plot!(sol_seir.t, R_seir_vals, label = "R", linewidth = 2, color = :green)
xlabel!("Time")
ylabel!("Fraction of population")
title!("Clustered SEIR (κ_s=3, κ_t=1, σ=0.2, anchored β, γ)")
```

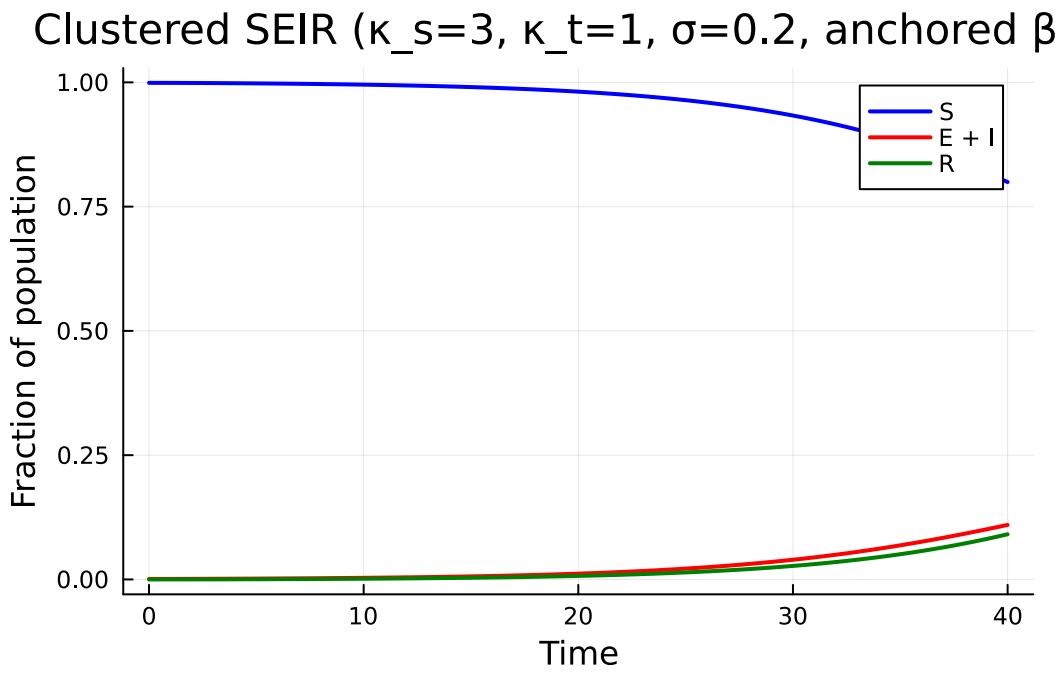

Figure 4: SEIR epidemic on a clustered network with a latent period (σ=0.2).

The latent period further delays and flattens the epidemic curve on top of the clustering effect.

Summary

- Clustering (triadic closure) is a key structural property of real contact networks that is absent from the standard configuration model.

- Triangles create redundant transmission paths: if two of three triangle edges transmit, the third cannot cause a new infection.
- The clustered EBCM uses a bivariate PGF $g(x, y)$ and tracks separate survival probabilities for single-edge (θ_2) and triangle-edge (θ_3) partners.
- Increasing clustering reduces R_0 and lowers epidemic peak prevalence while keeping the mean degree fixed.
- The `EdgeBasedModels.jl` package provides `clustered_poisson_pgf`, `build_clustered_sir`, and `build_clustered_seir` for modelling clustered networks with arbitrary disease progressions.

Simulation validation

We validate both the unclustered and clustered EBCMs against direct SSA on graphs sampled from the matching configuration models. The unclustered baseline uses an Erdős–Rényi graph at mean degree $\kappa = 5$. The clustered network uses the Newman–Miller doubly Poisson construction: each node draws Poisson($\kappa_s = 3$) single-edge stubs and Poisson($\kappa_t = 1$) triangle-corner stubs, single stubs are randomly paired, and triangle stubs are grouped in threes (3 edges per triangle) — yielding total mean degree $\kappa_s + 2\kappa_t = 5$.

The SSA initial seed fraction is matched to the EBM default ($\varepsilon = 10^{-3}$) so that EBM and SSA share the same initial condition and any peak-time shift reflects genuine model difference, not seeding mismatch.

```
include("../_validation.jl")

prog_clust = DiseaseProgression(
    [DiseaseStage(:I; transmission_rate = β_val), DiseaseStage(:R)],
    [DiseaseTransition(:I, :R, γ_val)]; entry = :I)

N_sim = 3000
n_g, n_s = 4, 25  # 100 SSA trajectories per treatment
tg = collect(0.0:0.5:40.0)
comps_view = [:S, :I, :R]
seed_frac = 1e-3  # match EBM default ε

# Unclustered SSA features and ribbon
F_un = ssa_feature_matrix(poisson_graph_builder(N_sim, 5.0),
    prog_clust, Dict(:β ⇒ β_val, :γ ⇒ γ_val), comps_view, tg;
    N = N_sim, n_graphs = n_g, nsims_per_graph = n_s,
    tspan = tspan, seed_fraction = seed_frac, base_seed = 20240801)

# Clustered SSA features and ribbon
```

```
F_cl = ssa_feature_matrix(clustered_poisson_graph_builder(N_sim, 3.0, 1.0),
    prog_clust, Dict(:β ⇒ β_val, :γ ⇒ γ_val), comps_view, tg;
    N = N_sim, n_graphs = n_g, nsims_per_graph = n_s,
    tspan = tspan, seed_fraction = seed_frac, base_seed = 20240802)

# Per-compartment ribbons (mean ± 1σ at each tgrid point), restricted to
# trajectories that produced a major outbreak so stochastic die-outs at
# this small seed fraction don't bias the SSA mean downward.
ssa_ribbon(F, comp_idx; r_idx = 3, r_thresh = 0.05) = begin
    nT = length(tg)
    R_final = F[:, r_idx * nT]
    keep = R_final .≥ r_thresh
    sub = F[keep, (comp_idx - 1) * nT + 1 : comp_idx * nT]
    vec(mean(sub; dims = 1)), vec(std(sub; dims = 1))
end
mu_un_S, sd_un_S = ssa_ribbon(F_un, 1)
mu_un_I, sd_un_I = ssa_ribbon(F_un, 2)
mu_un_R, sd_un_R = ssa_ribbon(F_un, 3)
mu_cl_S, sd_cl_S = ssa_ribbon(F_cl, 1)
mu_cl_I, sd_cl_I = ssa_ribbon(F_cl, 2)
mu_cl_R, sd_cl_R = ssa_ribbon(F_cl, 3)
```

```
([0.0, 0.00017753623188405796, 0.0003514492753623188, 0.0006014492753623188, 0.000869565217
```

```
p_un = plot(sol1.t, S1, label = "S (EBM)", lw = 2, color = :blue)
plot!(p_un, sol1.t, I1, label = "I (EBM)", lw = 2, color = :red)
plot!(p_un, sol1.t, R1, label = "R (EBM)", lw = 2, color = :green)
plot!(p_un, tg, mu_un_S, ribbon = sd_un_S,
      label = "S (SSA)", color = :blue, fillalpha = 0.2, linealpha = 0.6, lw = 1)
plot!(p_un, tg, mu_un_I, ribbon = sd_un_I,
      label = "I (SSA)", color = :red, fillalpha = 0.2, linealpha = 0.6, lw = 1)
plot!(p_un, tg, mu_un_R, ribbon = sd_un_R,
      label = "R (SSA)", color = :green, fillalpha = 0.2, linealpha = 0.6, lw = 1)
ylabel!(p_un, "Fraction of population")
title!(p_un, "Unclustered (ER, κ=5)")

p_cl = plot(sol2.t, S2, label = "S (EBM)", lw = 2, color = :blue)
plot!(p_cl, sol2.t, I2, label = "I (EBM)", lw = 2, color = :red)
plot!(p_cl, sol2.t, R2, label = "R (EBM)", lw = 2, color = :green)
plot!(p_cl, tg, mu_cl_S, ribbon = sd_cl_S,
      label = "S (SSA)", color = :blue, fillalpha = 0.2, linealpha = 0.6, lw = 1)
```

```
plot!(p_cl, tg, mu_cl_I, ribbon = sd_cl_I,
      label = "I (SSA)", color = :red, fillalpha = 0.2, linealpha = 0.6, lw = 1)
plot!(p_cl, tg, mu_cl_R, ribbon = sd_cl_R,
      label = "R (SSA)", color = :green, fillalpha = 0.2, linealpha = 0.6, lw = 1)
xlabel!(p_cl, "Time")
ylabel!(p_cl, "Fraction of population")
title!(p_cl, "Clustered (Newman–Miller, κ_s=3, κ_t=1)")

plot(p_un, p_cl, layout = (2, 1), size = (700, 600))
```

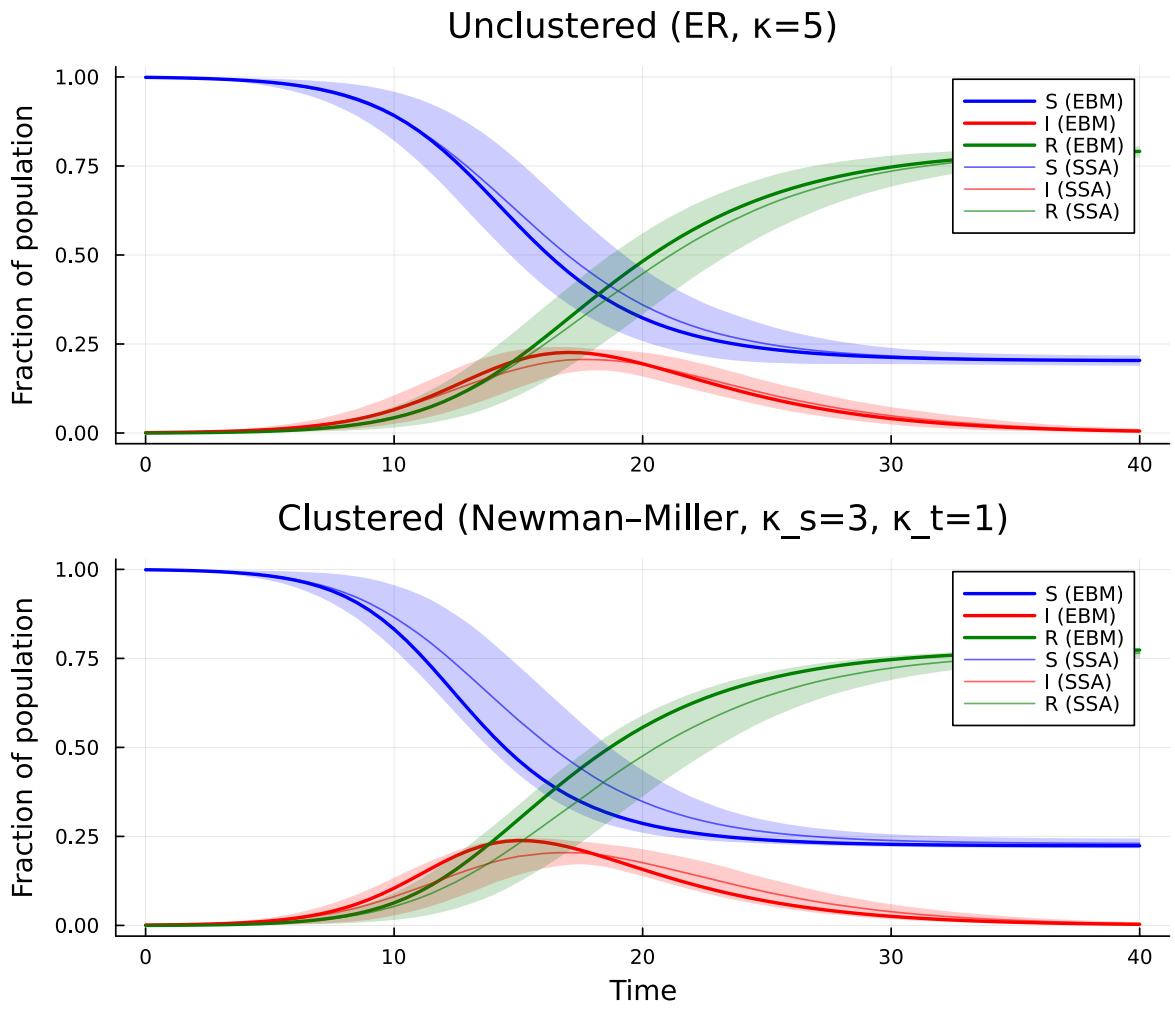

Figure 5: Two-panel comparison of EBM (lines) versus DirectSSA (ribbons = mean ± 1σ across 100 trajectories on 4 graphs of N=3000). Top: unclustered Erdős–Rényi network at mean degree 5. Bottom: Newman–Miller doubly Poisson clustered network with κ_s=3, κ_t=1 (same total mean degree).